

Improving Long-Rollout Accuracy of the DNO Surrogate

A status report on training, diagnostics, and architectural directions

Jonathan Ma

4 June 2026

Abstract

We report on a focused effort to close the long-rollout accuracy gap of a 1D Zakharov water-wave neural surrogate. Two training-time stability methods have been implemented and partly evaluated: a one-step pushforward loss (40-epoch run complete, an 80-epoch matched-compute resume in progress) and a Hamiltonian-conservation loss (Modal training in progress). Forensic analysis on the pushforward checkpoint vs. the FNO baseline identifies a coherent, linear-in- t in-band failure mode, strongly predicted within each regime by initial-condition bandwidth and maximum steepness (ρ up to +0.85). A 1000-snapshot mmap'd audit of the 73.7 GB training dataset finds no data-coverage problem: every test regime's initial conditions fall inside the training central 90% at the median. We summarise the empirical status of both training methods, characterise the failure mode in terms of physical descriptors of the initial condition, and rank concrete architectural and training-loss changes by expected gain. The single largest unexploited lever appears to be encoding the polynomial-in- η structure of the Craig–Sulem expansion into the architecture itself; longer-horizon unrolled training with gradients through all steps is the most straightforward training-side win.

1 Background and scope

The 1D Zakharov system on a periodic domain of length L couples the free-surface elevation $\eta(x, t)$ to the surface velocity potential $\xi(x, t)$ via the Dirichlet–Neumann operator $\mathcal{G}(\eta; h)$:

$$\partial_t \eta = \mathcal{G}(\eta; h) \xi, \tag{1}$$

$$\partial_t \xi = -g \eta - \frac{1}{2} \xi_x^2 + \frac{1}{2} \frac{(\mathcal{G}(\eta) \xi + \eta_x \xi_x)^2}{1 + \eta_x^2}. \tag{2}$$

Two neural surrogates are trained to approximate $(\eta, \xi, h) \mapsto \mathcal{G}(\eta; h) \xi$ on single snapshots and then iterated by an explicit time integrator at rollout time:

- **FNO baseline**, a 6-block FNO1d with width 128, modes 64, FiLM depth conditioning, soft-gating channel MLPs, and a linear-DNO baseline added in the output ($\sim 13.1\text{M}$ params).
- **SpectralDNO**, a 6-block model with width 128, modes 64, latent 64, concatenating $\log h$ as a feature channel and producing the residual as a rank-latent spectral sandwich $C(\eta, h)^\top F(\eta, h) C(\eta, h) \xi$ added to the linear baseline ($\sim 17.0\text{M}$ params). The output is *linear in ξ by construction*.

On the standard 10-regime evaluation suite (Tanaka solitons, Benjamin–Feir breathers, Stokes waves, random sea, linear) both surrogates are pointwise accurate but exhibit significant error growth over long autoregressive rollouts. This report covers the work undertaken to diagnose and mitigate the rollout error, and the recommended next steps.

2 Training-time stability methods

Both methods augment a baseline single-step Sobolev- H^k loss with an additional term targeting rollout stability. They are composable in the training script but the currently in-flight runs use one or the other.

2.1 Pushforward training

Given a data point (η, ξ) , advance the surrogate’s own prediction through k Euler steps of the Zakharov system, then re-predict $\mathcal{G}$ at the rolled-out state and penalise it against the analytical Craig–Sulem expansion at order four:

$$\mathcal{L} = \mathcal{L}_{\text{single-step}} + \lambda_{\text{pf}} \mathcal{L}(\mathcal{G}_{\theta}(\text{sg}(\eta_k), \text{sg}(\xi_k)), \mathcal{G}_{\text{CS}}(\eta_k, \xi_k)).$$

Gradient flow is restricted to the final $\mathcal{G}$ -prediction at the rolled-out state; the trajectory and the analytical target are both `stop_gradient`-wrapped. The currently active configuration uses $k = 1$, $\lambda_{\text{pf}} = 1$, $\text{dt} = 0.01$, and matches the model’s depth saturation $h_{\text{max}} = 5$ when constructing the analytical target.

Intent. Breaks the covariate-shift compounding of pure single-step training: the surrogate must remain accurate on states that it itself produces under iteration, not only on data-distribution states.

Cost. $(1 + k)$ forward passes plus one vmapped Craig–Sulem series evaluation per training step.

2.2 Hamiltonian-conservation loss

The Zakharov system is Hamiltonian with

$$H(\eta, \xi) = \frac{1}{2} \int_{\Omega} (\xi \mathcal{G}(\eta) \xi + g \eta^2) dx.$$

An exact solution conserves H . The added loss penalises clipped relative drift of H over one Euler step of the surrogate’s own dynamics:

$$\mathcal{L}_H = \lambda_H w(s) \mathbb{E} \left[\min \left(\frac{|H_1 - H_0|}{|H_0| + \varepsilon}, C \right) \right],$$

with H_0 evaluated at the data state, H_1 at one Euler step, both $\mathcal{G}$ values from the surrogate, sg on the advanced state, and a linear warmup $w(s) = \min(s/s_{\text{warm}}, 1)$. Configuration: $\lambda_H = 1$, $C = 10$, $s_{\text{warm}} = 10\,000$, $\text{dt} = 0.01$.

Intent. A physically motivated soft constraint: predictions that violate energy conservation are penalised in proportion to how much they violate it. Single-step horizon, so it does not stabilise long rollouts directly the way pushforward does.

Asymmetry. The pushforward branch mirrors the model’s depth saturation when building the Craig–Sulem target; the H-cons branch does not apply the clip when computing H or the Zakharov RHS. The relative-drift normalisation also makes the spatial measure $\text{d}x_{\text{phys}}$ cancel, so the spatial integration factor is effectively dead code in the current functional.

2.3 Composition with model saturation

The model itself clips $\log h$ at $\log 5$ so that $\tanh(hk) \rightarrow 1$ for $k \geq 1$. The pushforward target respects this clip; the H-cons branch does not. Either method can be disabled by setting its weight to zero.

3 Evaluation framework

3.1 Regimes

The eval suite covers ten regimes spanning depth (shallow/finite/deep), spectral bandwidth (monochromatic/broadband), and physical character (soliton/modulational/quasi-linear). Each provides 16 initial conditions and a rollout horizon $T_{\max}$ matching the integrator’s stable range.

3.2 Linear-regime fix

While diagnosing apparent surrogate failure on the “linear” regime, we discovered that the regime’s initial conditions are physically nonlinear shallow-water cases (η/h up to 0.30, $kh \approx 0.6$) inherited from the MATLAB legacy test set. The Zakharov solver used to produce the “ground truth” was diverging on some of these. We replaced the truth integrator with analytic per-mode Fourier propagation of the *linear* Zakharov system,

$$\omega^2(k) = g k \tanh(hk), \quad \hat{\eta}(k, t) = \hat{\eta}_0(k) \cos \omega t + \frac{\mathcal{G}_{\text{lin}}(k)}{\omega} \hat{\xi}_0(k) \sin \omega t,$$

and analogously for $\hat{\xi}(t)$. Amplitude and top- k structure are preserved to better than 1% over the full horizon. All linear-regime comparisons in this report use the analytic truth.

3.3 Metrics

We use per-IC relative L^2 error of η , $\|\eta_{\text{pred}}(t) - \eta_{\text{truth}}(t)\|_2 / \|\eta_{\text{truth}}(t)\|_2$, reported at three time slices ($T_{\max}/10$, $T_{\max}/2$, $T_{\max}$) or as full curves. ICs whose truth or prediction is non-finite anywhere in the rollout are flagged and counted separately rather than imputed.

4 Empirical findings

4.1 Pushforward training: a real but uneven win

Comparing FNO baseline (80 epochs) to SpectralDNO + pushforward (40 epochs), median rel- L^2 at the final time horizon:

Regime	FNO med	DNO+PF med	Δ	NaN ICs (FNO / DNO+PF)
tanaka_g0	0.246	0.099	−60%	0 / 1
tanaka_g1	0.605	0.255	−58%	0 / 1
bf_g0	0.174	0.130	−25%	0 / 3
bf_g1	0.193	0.130	−33%	1 / 2
bf_modal	0.245	0.127	−48%	3 / 2
linear (analytic)	1.170	0.341	−71%	0 / 4
stokes_deep	0.0036	0.0010	−72%	0 / 0
stokes_finite	0.0035	0.0056	+60%	0 / 0
random_sea_deep	0.0041	0.120	+2799%	0 / 0
random_sea_finite	0.0042	0.091	+2088%	0 / 0

Pushforward training delivers substantial slope reductions (5–25 $\times$) on long-horizon, broadband, shallow-water regimes (Tanaka, BF, modal, linear), but *regresses substantially* on random-sea regimes where the FNO baseline was already near-zero. NaN rates also rise under DNO+PF in some shallow regimes (e.g. 4 of 16 linear ICs blow up under DNO+PF; 0 under FNO).

Confound. The DNO+PF run is 40 epochs vs. 80 for the FNO baseline, with an architecture that has $\sim 30\%$ more parameters. A matched-compute 80-epoch DNO+PF resume run (with reset

Adam moments because of an optax pytree-shape mismatch on the saved opt-state) is in progress as of the cutoff for this report and will form the apples-to-apples comparison.

4.2 Failure mode is coherent in-band, not high- k

Across the three worst finite-error ICs (one each from linear, bf_modal, bf_g1) the error grows linearly or with slight super-linearity in time, r_{lin}^2 between 0.53 and 0.93. Decomposing the error spectrum into a peak band (centered on the truth’s dominant wavenumber) versus a high- k band, the per-IC cosine similarity between the error spectrum and the truth spectrum is 0.57–1.00 for FNO, with high- k Gibbs-style energy fraction below 0.18 in every worst case. NaN blow-ups in bf_g1 and bf_modal occur at $t = 81$ –189 out of $T_{\text{max}} = 200$ while $|\eta|_{\text{max}}$ stays small (0.04–0.12), i.e. numerical divergence after extended coherent drift, not amplitude blowup.

Specifically. On the linear regime, FNO at $t = 2$ has 99% of its error power in sideband modes $k = 10$ –50 while the truth has all its power at $k = 6$. This is sideband hallucination, and is the architectural cost of FNO mixing ξ into nonlinear features. By construction the SpectralDNO cannot generate this kind of error on the linear regime — consistent with its substantial improvement there.

4.3 Physics descriptors that predict failure

Within-regime Spearman correlations between IC physical descriptors and final-time $\text{rel-}L^2$:

- Tanaka g0/g1: spectral **bandwidth** $\rho = +0.76 / +0.78$ for FNO, $+0.84 / +0.85$ for DNO+PF — strongest signal in the dataset.
- Random-sea (deep): **max steepness** $\rho = +0.63$ (FNO) / $+0.84$ (DNO+PF).
- Stokes (finite): **max steepness** $\rho = +0.70 / +0.83$.
- BF g0/g1: k_{peak} $\rho \in [+0.33, +0.87]$.

Global correlations with kh and depth ($\rho \approx -0.4$) appear strong but are confounded — Tanaka has both shallow water and broad spectra, so the marginal global view conflates two distinct effects.

Read-off. The single most informative descriptor of where the surrogate fails is *not* a distributional one (kh , depth) but a *structural* one (bandwidth, steepness). High-steepness and broadband ICs need the nonlinear-in- η part of $\mathcal{G}$ to be accurate, which is exactly the part of the architecture we suspect is under-parameterised.

4.4 Rollout error vs. initial wavenumber

The headline diagnostic plot (Fig. 1) takes three ICs per regime binned by initial mean wavenumber and plots, on shared time axes, the relative L^2 error growth and the mean wavenumber of the truth, $\langle k \rangle(t) = \sum_k k |\hat{\eta}(k, t)|^2 / \sum_k |\hat{\eta}(k, t)|^2$.

The pattern visible in Tanaka g0 holds qualitatively in most regimes: higher- k ICs produce larger and faster-growing rollout error, while the truth’s mean wavenumber is essentially stationary (for the linear regime, exactly so). This rules out spectral drift of the underlying wave as an explanation for the error growth.

5 Data quality audit

A 1000-snapshot mmap’d sample of the 73.7 GB `combined_dataset_v3.npz` found no NaN/inf in over 10^6 sampled voxels of η , ξ , or $\mathcal{G}\xi$. Every test regime’s IC distribution falls inside the training central 90% at the median for amplitude, steepness, and dominant wavenumber. Two tails are worth noting:

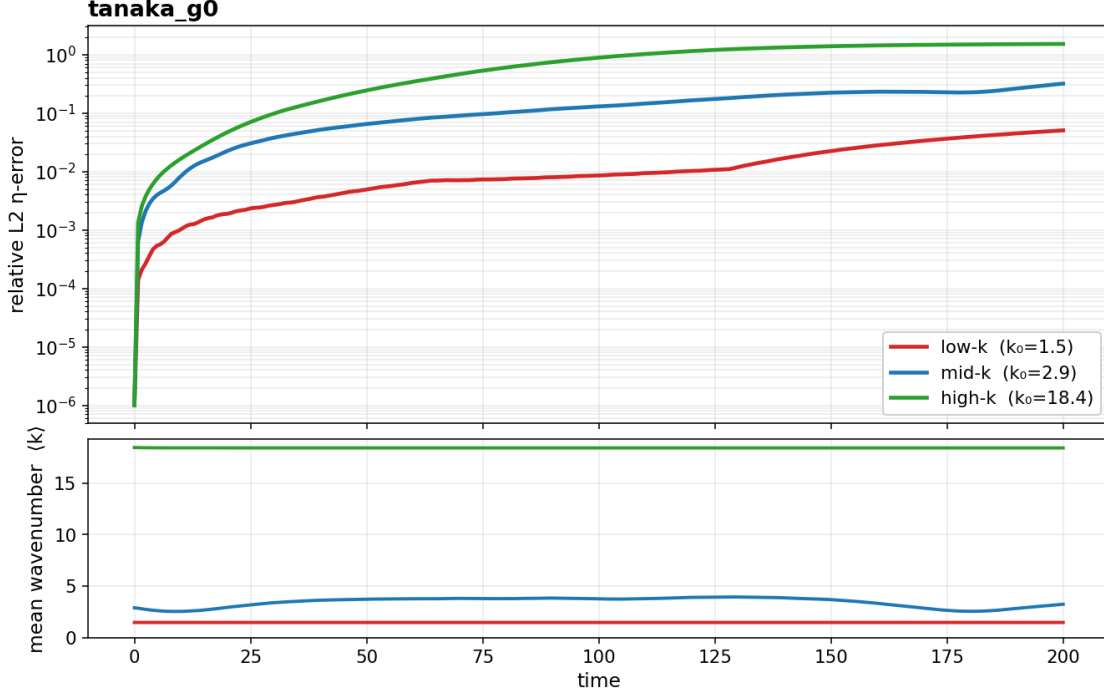


Figure 1: Tanaka g0: error growth and truth mean wavenumber vs. time for three ICs binned by initial mean wavenumber (red = low- k , blue = mid- k , green = high- k). Top: log-scale rel- L^2 η -error. Bottom: $\langle k \rangle(t)$ of the truth. The wave’s mean wavenumber is essentially stationary; the rollout error grows by roughly two orders of magnitude over the horizon, more rapidly for higher- k ICs.

- The maximum-amplitude Tanaka cases nip the training $p99$ in η and ξ amplitude.
- The `stokes.finite` dominant wavenumber sits at the upper edge of training k (~ 26); modes are still within the network’s modes = 64 cutoff.

One eval-side issue surfaced: one of the 16 `bf_g1` ICs has the *truth* integrator diverge to NaN at $t \approx 146$ of $T_{\max} = 200$. This pollutes any aggregate metric on that regime. It is not a training-data problem.

Conclusion. The rollout-accuracy ceiling is not a data-coverage problem. Improvements have to come from the model or the training-time loss, not from acquiring more data.

6 Architectural analysis

Both surrogates output

$$\mathcal{G}_\theta(\eta; h)\xi = G_0(h)\xi + R_\theta(\eta, h)\xi,$$

where $G_0(h) = |D| \tanh(h|D|)$ is the flat-surface linear DNO computed in closed form and R_θ is the learned residual.

- In FNO, R_θ is the standard FNO trunk applied to $(\eta, \xi, \partial\eta, \partial\xi, \dots)$ with FiLM depth conditioning; the trunk mixes ξ into the nonlinear features.
- In SpectralDNO, $R_\theta = C^\top(\eta, h)F(\eta, h)C(\eta, h)$ is a rank-latent spectral sandwich. By construction R_θ is *linear* in ξ , and the spectral filter $F(k)$ is constructed from a *mean-pooled* trunk feature — so the filter is one shape per IC, not spatially varying.

Two architectural observations follow directly from §4.2:

Linearity in ξ is a real win on the linear regime. FNO’s sideband hallucination on the linear regime is a structural consequence of mixing ξ into nonlinear features. SpectralDNO cannot produce this kind of error, and indeed dominates FNO on the linear regime by -71% in median $\text{rel-}L^2$.

The polynomial-in- η structure is under-parameterised. The Craig–Sulem expansion $\mathcal{G}(\eta)\xi = G_0(h)\xi + G_1(\eta, h)\xi + G_2(\eta \otimes \eta, h)\xi + \dots$ has G_n multilinear in $\eta^{\otimes n}$ and linear in ξ . SpectralDNO’s single learned sandwich is shape-compatible with one G_1 contribution; the architecture has no mechanism to express genuine G_2 or higher contributions, which are exactly the terms that dominate when steepness or bandwidth is large.

7 Recommended directions, ranked

We rank by expected gain $\times$ implementation effort, and distinguish architectural changes from training-loss changes.

7.1 Architecture

1. Polynomial-in- η block structure (Craig–Sulem-shaped). Replace the single learned sandwich with a stack of blocks each shaped like a learned G_n :

$$\text{DNOBlock}_n(\eta, \xi, h) \sim M_{\text{out}}(D, h) \sum_i [\Phi_i(\eta) \cdot M_i(D, h) \xi],$$

where Φ_i are learned spatial features of η (e.g. pointwise polynomials or small spatial MLPs) and $M_i(D, h)$ are learned depth-conditioned Fourier multipliers. Sum the outputs of 3–5 blocks and add to $G_0\xi$. The whole architecture is then *linear in ξ by construction* (preserving the win on the linear regime) and *polynomial in η by construction* (addressing the high-steepness failure mode). Theoretical grounding: this *is* the Craig–Sulem series with learned multipliers. Estimated implementation: ~ 150 lines of model code; no training-loop changes.

2. Depth-and- k -aware spectral filter. The current spectral filter is $F = \text{softplus}(\text{MLP}(\text{mean_pool}(\text{trunk})))$, with no explicit access to k or h . Replace with

$$F(k, h, \text{features}) = \text{softplus}(\text{MLP}(k, h, \tanh(hk), k \tanh(hk), \text{features})).$$

The first four inputs are the natural features of the dispersion relation; the MLP only needs to learn perturbations on this ansatz. Probably the highest gain-per-line change available; ~ 20 lines.

3. Spectral derivatives and half-derivative features. Replace the finite-difference $\partial_x, \partial_{xx}\eta$ features with spectral derivatives, and add $|D|^{1/2}\eta$ and the Hilbert transform $\mathcal{H}\eta$ as input features. $|D|^{1/2}$ is the natural scaling that appears in G_1 for deep water; currently the network must reconstruct it from local stencils. ~ 30 lines, no parameter increase.

Smaller wins. Smooth depth saturation ($h_{\text{eff}} = h_{\text{max}} \tanh(h/h_{\text{max}})$ rather than hard clip); port FNO’s per-mode weight-norm and learnable gain to SpectralDNO’s spectral conv; allow the spectral filter to be spatially varying via attention pooling rather than mean pooling.

7.2 Training-time

4. K-step unrolled training with gradients through all steps, $k = 4-8$, with curriculum.

List, Chen, Bali & Thuerey ([arXiv:2402.12971](#)) show that stop-gradient pushforward has a bias scaling like k^2 relative to the true multi-step gradient, and that $k \leq 4$ is often insufficient. Replace $\text{loss} = \text{MSE}(\text{model}(\text{sg}(\text{model}(u))), \text{target})$ with for $i \in [1, k]$: $u \leftarrow \text{model}(u)$; $\text{loss} += w_i \text{MSE}(u, \text{target}_i)$, no stop-gradient, and a curriculum that ramps k from 1 to 8 over training. Memory scales linearly in k ; at width 128, modes 64, $k = 4$ is comfortable on a single H100 with halved batch size.

5. Predict tendency, integrate externally with RK4. Bryutkin & Karniadakis ([arXiv:2412.13074](#))

report large rollout improvements from reformulating the target as $\partial_t u$ and using a real RK4 step at inference. Strictly speaking the DNO output is already a tendency (it is $\partial_t \eta$), so the analogous reformulation would be on the ξ side: have the model also predict the Bernoulli RHS rather than relying on a hand-coded formula at rollout. This decouples the surrogate from carrying the dispersive phase rotation across one large step. Modest implementation effort; high theoretical motivation.

6. PDE-Refiner-style noise-conditioned refinement. Lippe et al. ([arXiv:2308.05732](#)): standard MSE training neglects low-amplitude high- k modes, exactly the modes that drive modulational instability in Zakharov. Replace one-shot next-step prediction with $K = 3$ iterative noise-conditioned denoising passes. Inference cost $K\times$; reported gain is smaller on FNO than U-Net so the SpectralDNO is a better candidate here.

7.3 Skip list

Strict-symplectic / hard-Hamiltonian architectures (wrong constraint level for this discretisation), global Lipschitz bounds on the operator (kills the modulational dynamics for dispersive PDEs), simple capacity scaling (current architecture is 17M params vs. FNO’s 13M; capacity is not the bottleneck). See the literature review ([playground/diagnostics/literature_review.md](#)) for full reasoning on skipped directions.

8 Open questions

- Does the random-sea regression under DNO+PF survive the 80-epoch matched-compute resume, or is it a training-budget artifact?
- Is the H-conservation loss as currently written — evaluating H_0 with the *surrogate’s* $\mathcal{G}$ -prediction rather than the data label — the intended functional, or should H_0 use the true $\mathcal{G}\xi$ label?
- Does a Craig–Sulem-shaped architecture without pushforward already beat the current pushforward DNO? If so, the polynomial structure is the operative inductive bias, not the training loss.
- Per-IC NaN traces in `bf_g1/modal`: does divergence originate in the $\mathcal{G}$ -prediction or in the ξ -side Bernoulli RHS? Knowing this changes which loss term to weight.

9 Status of in-flight work

- **DNO+PF 80-epoch resume:** relaunched after an optax opt-state pytree-shape mismatch on the first attempt; running with `--reset_opt_state` (Adam moments rewarm in ~ 100 steps, negligible against 40 epochs). As of writing, epoch 2/40 complete, `val_loss` 0.00566. Rewarm is consistent with the original best 0.00490 at epoch 31.

- **H-cons training (Modal, L40S):** launched with linear warmup and per-sample clip; first evaluation pending.
- **Eval suite:** run on FNO baseline and DNO+PF 40-epoch; all 10 regimes evaluated, trajectories stored under `outputs/<run>/eval_suite/*_trajs.npz`. Will be re-run on the resumed and H-cons checkpoints when they finish.

Artifacts

- Data audit: `playground/diagnostics/data_audit_report.md`, `data_audit.json`, `audit_data.py`.
- Rollout forensics: `playground/diagnostics/rollout_forensics_report.md`, `rollout_forensics.json`, and the analysis scripts `rollout_forensics*.py`.
- Literature scan: `playground/diagnostics/literature_review.md`.
- Per-regime error-vs-wavenumber plots: `outputs/error_vs_k/*_err_vs_k.png`; site builder at `playground/build_paper_dashboard.py`; live at stunning-cocada-db3a7d.netlify.app.
- Linear-regime fix: `solver/evals/eval_suite.py`, function `truth.rollout_linear_analytic`.
- Pushforward and H-cons training implementation: `train-jax-10m/1d_dno_fno_jax.py`, lines 396–463 (loss assembly), 344–367 (Zakharov Euler step and analytical target wrapper).
- Companion slides: `notes/rollout_stability_approaches.tex/.pdf`.